

JAVA COLLECTION FRAMEWORK

Important Methods by Interface – Assignment

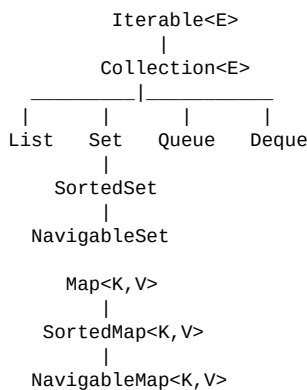
1. AIM

To study and demonstrate the important methods of the Java Collection Framework interfaces such as Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator and ListIterator.

2. DESCRIPTION

The Java Collection Framework provides interfaces and classes for storing and manipulating groups of objects. List maintains order and allows duplicates, Set stores unique elements, Queue follows FIFO behavior, Deque supports both ends, Map stores key-value pairs, and Sorted/Navigable interfaces provide ordering and navigation operations.

3. UML DIAGRAM



Iterator<E> ListIterator<E>

4. SOURCE CODE WITH COMMENTS

```
import java.util.*;

public class CollectionFrameworkDemo {
    public static void main(String[] args) {

        // Collection interface
        Collection<String> c = new ArrayList<>();
        c.add("A"); c.add("B"); c.add("C");
        System.out.println("Collection: " + c);
        System.out.println("Contains B: " + c.contains("B"));
        System.out.println("Size: " + c.size());

        // List interface - ordered and allows duplicates
        List<String> list = new ArrayList<>();
        list.add("C"); list.add("A"); list.add("B"); list.add("A");
        list.add(1, "D"); // Insert at index
        System.out.println("\nList: " + list);
        System.out.println("get(1): " + list.get(1));
        list.set(1, "E"); // Replace element
        System.out.println("After set: " + list);
        System.out.println("indexOf A: " + list.indexOf("A"));
        System.out.println("lastIndexOf A: " + list.lastIndexOf("A"));
        list.sort(Comparator.naturalOrder()); // Sort list
        System.out.println("Sorted List: " + list);

        // Set interface - duplicate elements are ignored
        Set<String> set = new HashSet<>();
        set.add("A"); set.add("B"); set.add("A");
        System.out.println("\nSet: " + set);
        System.out.println("Contains A: " + set.contains("A"));
        System.out.println("Set size: " + set.size());

        // SortedSet + NavigableSet
        NavigableSet<Integer> numbers = new TreeSet<>();
        numbers.add(10); numbers.add(20); numbers.add(30); numbers.add(40);
        System.out.println("\nNavigableSet: " + numbers);
        System.out.println("First: " + numbers.first());
        System.out.println("Last: " + numbers.last());
    }
}
```

```

System.out.println("lower(25): " + numbers.lower(25));
System.out.println("floor(20): " + numbers.floor(20));
System.out.println("ceiling(25): " + numbers.ceiling(25));
System.out.println("higher(30): " + numbers.higher(30));

// Queue interface - FIFO
Queue<String> queue = new LinkedList<>();
queue.offer("A"); queue.offer("B"); queue.offer("C");
System.out.println("\nQueue: " + queue);
System.out.println("peek(): " + queue.peek());
System.out.println("poll(): " + queue.poll());
System.out.println("Queue after poll: " + queue);

// Deque interface - insertion/removal at both ends
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("B"); deque.addLast("C"); deque.addFirst("A");
System.out.println("\nDeque: " + deque);
System.out.println("peekFirst(): " + deque.peekFirst());
System.out.println("peekLast(): " + deque.peekLast());
System.out.println("pollFirst(): " + deque.pollFirst());
System.out.println("Deque after pollFirst: " + deque);

// Map interface - key-value pairs
Map<Integer, String> map = new HashMap<>();
map.put(101, "A"); map.put(102, "B"); map.put(103, "C");
System.out.println("\nMap: " + map);
System.out.println("get(102): " + map.get(102));
System.out.println("containsKey(101): " + map.containsKey(101));
System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());

// NavigableMap
NavigableMap<Integer, String> nmap = new TreeMap<>();
nmap.put(10, "A"); nmap.put(20, "B");
nmap.put(30, "C"); nmap.put(40, "D");
System.out.println("\nNavigableMap: " + nmap);
System.out.println("lowerKey(25): " + nmap.lowerKey(25));
System.out.println("floorKey(20): " + nmap.floorKey(20));
System.out.println("ceilingKey(25): " + nmap.ceilingKey(25));
System.out.println("higherKey(30): " + nmap.higherKey(30));

// Iterator interface
System.out.print("\nIterator: ");
Iterator<String> it = list.iterator();
while (it.hasNext()) System.out.print(it.next() + " ");

// ListIterator - forward and backward traversal
System.out.print("\nListIterator forward: ");
ListIterator<String> lit = list.listIterator();
while (lit.hasNext()) System.out.print(lit.next() + " ");

System.out.print("\nListIterator backward: ");
while (lit.hasPrevious()) System.out.print(lit.previous() + " ");

System.out.println("\n\nCollection Framework demonstration completed.");
}

```

```

}

```

5. TERMINAL STYLE / OUTPUT SCREENSHOT

```
C:\Java> javac CollectionFrameworkDemo.java
C:\Java> java CollectionFrameworkDemo
```

```
Collection: [A, B, C]
Contains B: true
Size: 3
```

```
List: [C, D, A, B, A]
get(1): D
After set: [C, E, A, B, A]
indexOf A: 2
lastIndexOf A: 4
Sorted List: [A, A, B, C, E]
```

```
Set: [A, B]
Contains A: true
Set size: 2
```

```
NavigableSet: [10, 20, 30, 40]
First: 10
Last: 40
lower(25): 20
floor(20): 20
ceiling(25): 30
higher(30): 40
```

```
Queue: [A, B, C]
peek(): A
poll(): A
Queue after poll: [B, C]
```

```
Deque: [A, B, C]
peekFirst(): A
peekLast(): C
pollFirst(): A
Deque after pollFirst: [B, C]
```

```
Map: {101=A, 102=B, 103=C}
get(102): B
containsKey(101): true
Keys: [101, 102, 103]
Values: [A, B, C]
```

```
NavigableMap: {10=A, 20=B, 30=C, 40=D}
lowerKey(25): 20
floorKey(20): 20
ceilingKey(25): 30
higherKey(30): 40
```

```
Iterator: A A B C E
ListIterator forward: A A B C E
ListIterator backward: E C B A A
```

```
Collection Framework demonstration completed.
```

6. RESULT

Thus, the Java Collection Framework was successfully studied and implemented. The program demonstrated important methods of Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator and ListIterator.

Note: HashSet and HashMap do not guarantee a fixed iteration order, so their displayed order may vary.